

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁRSKA PRÁCA

Rozšířitelná knižnica pre správu Docker kontajnerov
implementovaná v Go



2026

Vedúci práce:
Mgr. Tomáš Urbanec, Ph.D.

Samuel Táboorský

Študijný program: Informatika,
Špecializácia: Programovanie a vývoj
software

Bibliografické údaje

Autor:	Samuel Tábořský
Názov práce:	Rozšířitelná knihovna pro správu Docker kontajnerů implementovaná v Go
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Študijný program:	Informatika, Špecializácia: Programovanie a vývoj software
Vedúci práce:	Mgr. Tomáš Urbanec, Ph.D.
Počet strán:	35
Prílohy:	elektronické dáta v úložisku katedry informatiky
Jazyk práce:	slovenský

Bibliographic info

Author:	Samuel Tábořský
Title:	An extensible library for Docker Container management implemented in Go
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Tomáš Urbanec, Ph.D.
Page count:	35
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Slovak

Anotácia

Táto bakalárska práca sa zaoberá návrhom a implementáciou knižnice v jazyku Go na správu Docker kontajnerov. Opisuje teoretické východiská kontajnerizácie a prístup cez Docker API, pričom sa sústreďuje na modulárnu architektúru umožňujúcu rozšíriteľnosť aj podporu rôznych kontajnerových behových prostredí. Praktická časť práce zahŕňa implementáciu knižnice a sprievodnej ukázkovej aplikácie pre správu kontajnerov, sledovanie udalostí a monitoring prostriedkov. Výsledné riešenie je overené testami a predstavuje základ pre ďalší vývoj a rozšírenie v open-source prostredí.

Synopsis

This bachelor's thesis focuses on the design and implementation of a Go-based library for Docker container management. It outlines the fundamentals of containerization and Docker API access, with emphasis on a modular architecture enabling extensibility and support for multiple container runtimes. The practical part includes the implementation of the library and an accompanying demo application for container management, event monitoring, and resource tracking. The solution is validated by testing and provides a foundation for further development and open-source extension.

Kľúčové slová: Docker; kontajnerizácia; knižnica; jazyk Go

Keywords: Docker; containerization; library; Go language

Najskôr by som chcel poďakovať pánovi Mgr. Tomášovi Urbancovi PhD. za jeho odborné vedenie a rady, ktoré mi poskytol počas písania tejto bakalárskej práce. Takisto by som rád poďakoval mojej rodine a blízkym za ich veľkú podporu počas môjho štúdia.

Odovzdaním tohto textu jeho autor čestne vyhlasuje, že celú prácu vrátane príloh vypracoval samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1	Úvod	8
1.1	Motivácia a ciele práce	8
1.2	Koncept riešenia	9
1.3	Prehľad existujúcich riešení	9
1.4	Prínos a očakávania	10
2	Teoretické základy	11
2.1	Virtualizácia a kontajnerizácia	11
2.2	Docker a jeho architektúra	11
2.3	Programovací jazyk Go a jeho ekosystém	12
2.4	Softvérové architektúry a rozšíriteľnosť knižníc	13
3	Analýza problému	13
3.1	Požiadavky na systém	13
3.1.1	Funkčné požiadavky	14
3.1.2	Nefunkčné požiadavky	14
3.2	Prehľad dostupných knižníc a nástrojov	15
3.2.1	Komunikácia s Docker Engine: REST API a SDK	15
3.2.2	Používané knižnice a technológie	15
3.2.2.1	Moby Project a oficiálne Go SDK	16
3.2.2.2	Nástroje pre budúcu rozšíriteľnosť	16
3.2.2.3	Pomocné knižnice ekosystému Go	16
4	Návrh a implementácia riešenia	16
4.1	Návrh architektúry knižnice	17
4.1.1	Štruktúra balíčkov a repozitára	17
4.1.2	Návrh verejného rozhrania	18
4.1.3	Implementačná vrstva a vzor Adaptér	19
4.2	Implementácia Docker klienta	19
4.2.1	Inicializácia klienta a bezpečnosť vlákien	19
4.2.2	Transformácia objektov a mapovacie funkcie	20
4.2.3	Asynchrónne spracovanie telemetrie a udalostí	20
4.3	Testovanie a validácia	21
4.3.1	Jednotkové testy	21
4.3.2	Testovanie pomocou falošných objektov	21
4.4	Implementácia ukázkovej aplikácie	22
4.4.1	Návrh a princíp fungovania	22
4.4.2	Využitie vlastností knižnice v praxi	23
5	Používateľská dokumentácia	25
5.1	Predpoklady a inštalácia	25
5.2	Inicializácia a základné nastavenie	25
5.3	Práca s Docker obrazmi	26

5.4	Správa životného cyklu kontajnerov	26
5.5	Odber logov a telemetrie	27
5.6	Spracovanie chýb	28
6	Možné budúce rozšírenia	28
6.1	Podpora ďalších behových prostredí	28
6.2	Správa zväzkov a sieťovanie	29
6.3	Open-source pokračovanie	29
	Záver	31
	Conclusions	32
	A Obsah elektronických dát	33
	Literatúra	34

Zoznam zdrojových kódov

1	Príklad použitia knižnice na vytvorenie a spustenie kontajneru. . .	18
2	Ukážka definície falošného klienta pre testovanie rozhrania. . . .	22
3	Vykonanie izolovanej úlohy s využitím knižnice.	24
4	Inštalácia knižnice.	25
5	Inicializácia klienta.	26
6	Ukážka stiahnutia obrazu.	26
7	Ukážka správy životného cyklu kontajneru.	27
8	Ukážka čítania telemetrie v reálnom čase.	27
9	Ukážka spracovania chýb.	28

1 Úvod

V posledných rokoch sa kontajnerizácia stala základným stavebným kameňom vývoja a nasadzovania moderného softvéru. Na tomto koncepte je postavené veľké množstvo moderných produktov a nástrojov, od cloudových platforiem použiteľných pre nasadenie softvéru do produkčného prostredia, nástrojov na jednoduché zdieľanie kompletných vývojových prostredí medzi softvérovými inžiniermi a programátormi, až po škálovateľné systémy založené na architektúre mikroslužieb (*microservice architecture*). Väčšina súčasných webových aplikácií a webstránok už je nasadených v distribuovanom systéme postavenom z kontajnerov, čo prináša možnosť veľmi jednoduchého škálovania, vyrovnávania záťaže (*load balancing*), zálohovania a zaručenia vysokej dostupnosti. Prechod z tradičnej hardvérovej virtualizácie na kontajnerizáciu uľahčil a zefektívnil každú časť softvérového vývojového procesu, počínajúc reprodukovateľnými vývojovými prostrediami, cez integráciu *Continuous Integration/Continuous Deployment* (ďalej len CI/CD) postupov do vývoja, až po nasadenie a správu v produkcii.

Jedna z najznámejších technológií spojených s kontajnerizáciou je *Docker*. Poskytuje komplexné rozhranie na správu životného cyklu kontajnerov a obrazov, ako aj na konfiguráciu sietí, pripájanie lokálnych zväzkov a monitorovanie udalostí. Docker však nie je jediná technológia v tomto ekosystéme. Okrem neho existujú ďalšie nástroje s rôznymi architektúrami. Medzi niektoré známe produkty a technológie patrí napríklad daemonless *Podman*, distribuované orchestračné platformy ako *Kubernetes*, alebo nízkoúrovňové behové prostredie *containerd*. Táto rôznorodosť prístupov a abstrakčných vrstiev so sebou prináša komplexné prostredie, s ktorým sa súčasní vývojári musia vysporiadať.

1.1 Motivácia a ciele práce

Pre lokálne vývojové prostredia vývojári často preferujú vysokoúrovňové riešenia (ako napríklad Docker), postavené na princípe jednoduchého používania a poskytujúce napríklad konzolové alebo grafické rozhranie. V produkčnom prostredí a pri orchestrácii sú však uprednostňované nástroje oveľa nižšej úrovne, ako napríklad ľahký *containerd*.

Tento fakt prináša rôzne problémy: diametrálne odlišné rozhranie a jeho zložitnosť, spôsob práce, a nutnosť ovládať niekoľko nástrojov. Kým knižnica pre Docker je postavená na jednoduchších akciách, *containerd* je omnoho modulárnejší a vyžaduje od vývojára manuálnu správu niekoľkých komponentov (napr. snapshoty, namespaces, tasky a iné).

Zároveň vyvstáva problém tzv. *Vendor lock-in*, čo v praxi znamená neprenositelnosť aplikácie, postavenej napríklad na Docker *Software Development Kit* (ďalej len SDK), na systémy, kde beží iba *containerd* alebo *Kubernetes*. Ak je žiadané podporovať niekoľko druhov systémov, je nutné udržiavať viaceré odlišné kusy kódu, čo prináša ďalšiu komplexnosť.

Mojou motiváciou bolo vytvoriť jednoduché a elegantné unifikované rozhra-

nie v jazyku Go, ktoré by vývojárom umožnilo použiť jeden kód na prácu s kontajnermi bez ohľadu na to, aký nástroj sa skrýva v pozadí. Cieľom tejto práce je analyzovať a porovnať existujúce riešenia na správu kontajnerov a ich architektúru. Na základe tejto analýzy potom navrhnúť jednotné rozhranie a abstrakčnú vrstvu, ktorá skryje implementačné rozdiely jednotlivých behových prostredí pred koncovým používateľom. Nasleduje implementácia samotnej knižnice v jazyku Go, ktorá obsahuje modul pre prácu s nástrojom Docker, a zároveň jasný a jednoduchý spôsob, ako knižnicu rozšíriť o ďalšie kontajnerové behové prostredia. Nakoniec vytvorenú knižnicu otestovať a pripraviť ukázkový program demonštrujúci jej použitie.

1.2 Koncept riešenia

Základom implementácie knižnice je jednotné rozhranie obsahujúce funkcie na správu životného cyklu kontajnerov a obrazov, konfiguráciu sietí, pripájanie lokálnych zväzkov a monitorovanie a reakcie na rôzne udalosti. Z nástrojov Docker, Podman a containerd bola vybraná čo najširšia spoločná funkcionálna, ktorá tvorí toto rozhranie. Zároveň knižnica obsahuje jednoduché dátové typy a štruktúry, ktoré odtienia konkrétne implementačné detaily, vstupné a návratové typy a spôsoby predávania argumentov funkciám jednotlivých technológií.

Vďaka tomu má vývojár dostupný nástroj, pomocou ktorého je možné implementovať aplikácie bez potreby písania kódu pre rôzne behové prostredia, pracovania s rozdielnymi dátovými typmi a spôsobmi riadenia toku programu. Takto implementovaná knižnica predstavuje základ pre jednoduchý, jednotný a rozširiteľný systém, ktorý umožní nielen vývoj platformovo nezávislých aplikácií, ale napríklad aj ich jednoduché testovanie. Vývojár má možnosť vytvárať a spravovať kontajnery so službami, ktoré sú potrebné počas behu testu, ako sú mimo iného databázové systémy alebo autentifikačné a logovacie nástroje.

1.3 Prehľad existujúcich riešení

Najčastejší prístup, akým vývojári momentálne integrujú do svojich projektov kontajnerové behové prostredia, je pomocou využitia ich oficiálnych klientských knižníc. Pre Docker existuje *Moby SDK* [1], pre Podman je dostupné *Podman SDK* [2] a pre containerd existuje oficiálna knižnica slúžiaca pre tvorcov iných nástrojov (napr. samotný Docker používa túto knižnicu) [3]. Tieto riešenia však neposkytujú žiadnu spoločnú abstrakciu, takže je nutné udržiavať samostatný kód a vetvy logiky pre každé z nich.

Projekt Kubernetes definoval pomocou gRPC protokolu spoločné rozhranie *Kubernetes Container Runtime Interface* (ďalej len CRI) [4]. Toto rozhranie funguje na princípe klient-server. Na každom serveri (node) toto rozhranie používa agent *Kubelet*. Ten pomocou neho komunikuje s nízkoúrovňovým engine (napr. containerd, *cri-o*, alebo v minulosti *dockershim*), ktorý toto rozhranie implementuje. CRI je síce extrémne robustné riešenie, ale je určené pre masívnu orches-

tráciu. To pre vývojára znamená implementáciu gRPC klienta a komunikáciu cez CRI, čo je pri písaní menších projektov, skriptovaní, automatizácii či vývoji *Command-line Interface* (ďalej len CLI) nástrojov príliš komplikované riešenie.

Ďalší populárny projekt v ekosystéme jazyka Go je *Testcontainers* [5]. Táto knižnica umožňuje vývojárom priamo v jednotkových testoch programaticky naštartovať napríklad databázu, spustiť testy a nakoniec kontajner zahodiť. Je však viazaná na kontext softvérového testovania. Obsahuje logiku pre čakanie na rôzne operácie (tzv. *waiting strategies*), automatické mazanie kontajnerov pomocou privilegovaného *Ryuk* kontajneru a iné nástroje mierené na testovanie softvéru. Nie je to všeobecná abstrakčná knižnica určená na produkčnú správu životného cyklu kontajnerov.

Podman poskytuje službu, ktorá emuluje Docker *Representational State Transfer Application Programming Interface* (ďalej len API), často na rovnakom sockete, čo umožňuje staršie nástroje napísané pre Docker používať s Podman. Toto však rieši problém fragmentácie rozhraní a dátových typov iba pre tieto dve kontajnerové behové prostredia, a to iba na úrovni operačného systému, nie v programovom kóde. Stále teda chýba nástroj, ktorým je možné jedným kódom ovládať napríklad Docker, Podman a containerd zároveň.

Z prehľadu existujúcich riešení vyplýva, že hoci existujú snahy o štandardizáciu (ako napríklad rozhranie CRI v Kubernetes), tieto riešenia sú pre mnohé menšie a stredne veľké softvérové projekty príliš robustné a ťažkopádne. Na druhej strane, priame použitie natívnych SDK knižníc vedie k duplicite kódu a silnej závislosti na konkrétnom poskytovateľovi. V súčasnosti v ekosystéme jazyka Go chýba ľahká, všeobecne použiteľná knižnica, ktorá by poskytla vývojárom jednotné rozhranie a abstrahovala by radikálne rozdiely medzi rozhraniami rôznych kontajnerových behových prostredí bez nutnosti implementácie robustných architektonických vzorov z prostredia Kubernetes.

1.4 Prínos a očakávania

Hlavným prínosom tejto práce je poskytnutie nástroja, ktorý odstráni závislosť aplikácií od jedného konkrétného kontajnerového behového prostredia. Vývojári, ktorí túto knižnicu použijú, nebudú pevne zviazaní napríklad iba s Docker, ale budú mať dostupný spôsob, akým rovnaký kód použiť nezávisle na systéme, na ktorom bude ich aplikácia nasadená, prípadne nezávisle na rozhodnutí zmeniť prostredie, ktorý beží v pozadí. Práca prinesie čisté, unifikované API v jazyku Go. Vďaka abstrakcii konkrétnych implementačných detailov jednotlivých kontajnerových prostredí sa rapídne zníži čas potrebný na vývoj nástrojov pracujúcich s kontajnermi. Zároveň práca ponúka jednoduché riešenie pre vývojárov, pre ktorých je implementácia komplexného CRI (Container Runtime Interface) cez gRPC protokol zbytočne náročná a neprimeraná ich potrebám.

Projekt obsahuje centrálny balíček obsahujúci unifikované rozhranie pre operácie nad kontajnermi a obrazmi, a zároveň model štruktúr, ktoré toto rozhranie využíva. Taktiež obsahuje balíček pre Docker na komunikáciu s Docker API.

2 Teoretické základy

Táto kapitola poskytuje nevyhnutný teoretický rámec pre pochopenie rozdielov medzi tradičnou virtualizáciou a modernou kontajnerizáciou, pričom sa detailne zameriava na architektúru platformy Docker. Zároveň analyzuje špecifiká programovacieho jazyka Go a princípy návrhu softvérových architektúr, ktoré sú kľúčové pre zabezpečenie vysokej rozšíriteľnosti a modularity výslednej knižnice.

2.1 Virtualizácia a kontajnerizácia

S pokrokmi v softvérovom inžinierstve sa základnou výzvou stalo zabezpečenie spoľahlivého a konzistentného nasadenia aplikácií v rôznych výpočtových prostrediach. Historickým pokrokom v tomto ohľade bola virtualizácia prostredníctvom virtuálnych strojov (*VM - Virtual Machines*). Toto riešenie je postavené na softvérovej vrstve zvanej hypervízor, ktorý emuluje hardvér stroja. Na takto emulovanom hardvéri je spustený kompletný operačný systém úplne izolovaný od hostiteľského operačného systému a iných virtuálnych strojov. Výhoda tohto prístupu je vysoká úroveň bezpečnosti. Medzi nevýhody však patrí značná systémová réžia, keďže každý virtuálny stroj spotrebuje zdroje potrebné na beh operačného systému.

Kontajnerizácia je modernejší a efektívnejší prístup k izolácii aplikácií. Všetky kontajnery spustené na hostiteľskom operačnom systéme zdieľajú jeho jadro a sami neobsahujú vlastný operačný systém, ale iba samotnú aplikáciu a jej nevyhnutné závislosti. Tento model prináša značné zníženie nárokov na systémové zdroje, rýchlejšie spúšťanie kontajnerov a väčšie množstvo kontajnerov spustených na jednom fyzickom serveri. Technologický základ kontajnerizácie v Linuxe je postavený na dvoch funkciách jadra: *menné priestory (namespaces)* a *kontrolné skupiny (cgroups)*. Menné priestory zabezpečujú izoláciu - každý kontajner má svoj vlastný, oddelený pohľad na systémové zdroje. Kontrolné skupiny naopak slúžia na riadenie a obmedzovanie systémových zdrojov, ktoré môže kontajner využívať.

2.2 Docker a jeho architektúra

Docker je platforma s otvoreným zdrojovým kódom, ktorá abstrahuje tieto implementačné detaily a poskytuje ucelený ekosystém nástrojov na automatizáciu, nasadzovanie, spúšťanie a správu aplikácií bežiacich v kontajneroch. Vďaka svojej robustnosti a jednoduchosti sa postupne stal štandardom v oblasti kontajnerizácie a posunu k nasadeniam mikroslužieb.

Jadro platformy Docker je postavené na architektúre klient-server. Tá sa skladá z troch základných častí: perzistentný proces Docker (*dockerd*), rozhranie *REST API* a Docker klient (*docker*) [6]. Perzistentný proces beží v pozadí na hostiteľskom systéme a je zodpovedný za všetky kľúčové operácie. Svoje funkcie vystavuje pomocou *REST API* rozhrania, ktoré štandardne počúva na unixovom sokete. Slúži ako centrálny komunikačný bod pre akéhokoľvek klienta,

ktorý chce interagovať s procesom `dockerd`. Docker CLI je konzolový nástroj slúžiaci ako primárne rozhranie pre používateľa. Každý užívateľský príkaz preloží na *Hypertext Transfer Protocol* (ďalej len HTTP) požiadavok, odošle ho na Docker API, odkiaľ bude následne spracovaná a vykonaná.

Okrem týchto komponentov sú pre prácu s Docker kľúčové tieto objekty:

- **Obraz (Image):** Šablóna s inštrukciami na vytvorenie kontajneru. [7]
- **Kontajner (Container):** Spustiteľná inštancia obrazu. [8]
- **Register (Registry):** Vzdialené alebo lokálne úložisko pre Docker obrazy, ako napríklad Docker Hub. [9]

Praktický výstup tejto práce sa do tejto architektúry začleňuje ako programatický klient. Používateľovi umožňuje komunikovať s platformou Docker pomocou kódu aplikácie napísanej v jazyku Go a obísť tak integráciu s príkazovým riadkom.

2.3 Programovací jazyk Go a jeho ekosystém

Voľba programovacieho jazyku *Go* ako technologického základu pre túto prácu nebola náhodná. Go, jazyk vyvinutý spoločnosťou Google, bol navrhnutý s dôrazom na jednoduchosť, efektivitu a spoľahlivosť pri vývoji komplexných aplikácií, vďaka čomu dosiahol dominantnú rolu v oblasti *cloud-native* softvéru. Samotný Docker, ale aj Kubernetes a ďalšie nástroje v tomto ekosystéme sú napísané práve v Go. Ako kompilovaný a staticky typovaný jazyk poskytuje typovú bezpečnosť a vysoký výkon, zatiaľ čo kompilácia do jedného binárneho súboru bez externých závislostí značne zjednodušuje nasadenie a distribúciu aplikácií v Go.

Významnou prednosťou Go v kontexte tejto práce je jeho zabudovaný model súbežnosti *Communicating Sequential Processes (CSP)* [10] založený na dvoch kľúčových konceptoch [11]:

- **Goroutines (gorutiny):** Lhké vlákna spravované priamo behovým prostredím jazyka Go, nie operačným systémom. Ich vytváranie a správa majú malú réžiu, vďaka čomu je možný beh veľkého množstva takýchto vlákien v jednej aplikácii.
- **Channels (kanály):** Typovo bezpečný komunikačný mechanizmus, pomocou ktorého si gorutiny môžu bezpečne a synchrónne posilať dáta. Tento prístup pomáha predchádzať problémom so súbehom (*race conditions*) a uľahčuje návrh asynchrónnych operácií.

Praktická časť mojej práce tieto vlastnosti využíva pri implementácii funkcií spravujúcich dátové prúdy z Docker API. Monitorovanie prostriedkov využitých kontajnerom alebo odpočúvanie systémových udalostí na pozadí spúšťajú dedikovanú gorutinu, ktorá nepretržite dekoduje a spracúva dáta prichádzajúce z Docker API. Vďaka tomu môže používateľ pohodlne prijímať telemetriu a systémové udalosti v reálnom čase bez toho, aby došlo k zablokovaniu hlavného vlákna aplikácie.

2.4 Softvérové architektúry a rozšíriteľnosť knižníc

Kvalita softvéru sa neodvíja iba od jeho funkčnosti, ale aj od jeho vnútorného návrhu. Správna architektúra zaručuje ľahkú použiteľnosť, jednoduchosť testovania a rozšíriteľnosť o novú funkcionálnosť bez nutnosti prepisovania kódu.

Základným kameňom návrhu praktickej časti je programovanie voči rozhraniám (*interface-driven design*) a úzko súvisiaci princíp inverzie závislostí (*Dependency Inversion Principle*), piaty z princípov SOLID [12]. Hlavnou myšlienkou je znížiť pevnú väzbu medzi modulmi a urobiť systém flexibilnejším a rozšíriteľnejším. To je postavené na kvalitnom návrhu abstrakcií, od ktorých sú moduly závislé. Týmto abstrakciami sú v jazyku Go práve rozhrania. Výsledkom takéhoto návrhu je voľná väzba (*loose coupling*) medzi komponentami, kde zmena implementačných detailov jednej komponenty neovplyvní ostatné moduly, pokiaľ sú dodržané kontrakty definované rozhraním. To dramaticky zvyšuje flexibilitu a zjednodušuje testovanie, pretože pri jednotkových testoch je možné konkrétnu implementáciu API jednoducho nahradiť za falošnú (*mock*) a testovať tak komponentu v úplnej izolácii od behového prostredia.

Ďalším často využívaným architektonickým prvkom je návrhový vzor Adaptér (*Adapter*). Úlohou tohoto vzoru je obaliť existujúci, často zložitý alebo nekompatibilný systém a poskytnúť preň zjednodušené, čistejšie a účelovo zamerané API. Návrhový vzor Fasáda (*Facade*) potom skrýva vnútornú komplexnosť subsystému a vystavuje iba tie funkcie, ktoré sú pre klienta relevantné, čím znižuje kognitívnu záťaž vývojára a zabráňuje chybám plynúcim z nesprávneho použitia zložitého rozhrania. Aplikácia týchto princípov je základom pre tvorbu robustných a dlhodobo udržiavateľných softvérových riešení.

3 Analýza problému

V tejto časti sa zameriavam na hĺbkovú analýzu špecifikácií potrebných pre úspešnú implementáciu knižnice, pričom definujem kľúčové funkčné a nefunkčné požiadavky na systém. Súčasťou je aj kritický prehľad existujúcich technologických riešení, ktoré tvoria základ pre komunikáciu s behovým prostredím a zabezpečujú budúcu rozšíriteľnosť riešenia v rámci ekosystému Go.

3.1 Požiadavky na systém

Cieľom tejto bakalárskej práce je návrh a implementácia softvérovej knižnice v jazyku Go, ktorá poskytne vysokoúrovňové, jednoducho použiteľné a rozšíriteľné rozhranie pre programovú správu životného cyklu kontajnerov. Navrhovaná knižnica má za cieľ skryť komplexnosť a nízku úroveň abstrakcie existujúcich oficiálnych SDK a poskytnúť rozhranie, ktoré je intuitívne, typovo bezpečné a zamerané na najčastejšie prípady použitia. Na základe analýzy zadania boli definované nasledujúce funkčné a nefunkčné požiadavky na systém.

3.1.1 Funkčné požiadavky

Funkčné požiadavky definujú, aké konkrétne operácie musí systém umožňovať. Boli rozdelené do logických celkov podľa spravovaného typu zdrojov.

- **Správa životného cyklu kontajnerov:**
 - Systém musí umožniť vytvorenie, spustenie, zastavenie, pozastavenie, obnovenie, reštartovanie a odstránenie kontajnerov.
 - Musí poskytovať funkciu na výpis existujúcich kontajnerov s možnosťou filtrovania.
 - Musí umožniť inšpekciu konkrétneho kontajnera a získať jeho detailný stav.
 - Musí podporovať spustenie príkazov vo vnútri bežiaceho kontajneru.
 - Musí implementovať mechanizmus na čakanie, kým kontajner nedosiahne určitý stav.
 - Musí umožniť streamovanie logov z kontajneru v reálnom čase.
- **Správa obrazov:**
 - Systém musí zabezpečiť sťahovanie obrazov z registrov.
 - Počas sťahovania obrazu musí byť možné sledovať priebeh operácie.
 - Musí poskytovať funkcie na výpis a inšpekciu obrazov.
 - Musí umožniť odstránenie nepotrebných obrazov.
- **Monitorovanie udalostí a telemetrie:**
 - Systém musí poskytovať asynchrónny prúd udalostí z behového prostredia.
 - Musí umožniť streamovanie telemetrických dát o využití zdrojov (CPU, pamäť).

3.1.2 Nefunkčné požiadavky

Nefunkčné požiadavky definujú kvalitatívne vlastnosti a architektonické ciele systému.

- **Jednoduchosť a abstrakcia:** Rozhranie knižnice musí byť výrazne jednoduchšie na použitie ako natívne SDK.
- **Rozšíriteľnosť:** Architektúra musí byť navrhnutá tak, aby bolo v budúcnosti možné pridať podporu pre alternatívne behové prostredia.
- **Testovateľnosť:** Návrh systému musí umožňovať jednoduché písanie jednotkových testov.

- **Spracovanie chýb:** Knižnica musí mapovať širokú a často nekonzistentnú škálu chýb z nízkoúrovňového SDK na jasne definovanú sadu sémantických chýb.

3.2 Prehľad dostupných knižníc a nástrojov

Efektívna interakcia s Docker vyžaduje voľbu medzi priamou komunikáciou s REST API a využitím vysokoúrovňových abstrakcií, ktoré ponúka oficiálne SDK.

3.2.1 Komunikácia s Docker Engine: REST API a SDK

Docker Engine, ako bolo popísané v kapitole 2.2, funguje na princípe klient-server. Na komunikáciu s ním existujú dva základné prístupy: priame volanie REST API alebo použitie oficiálneho SDK.

- **REST API:** Predstavuje základné, jazykovo-agnostické rozhranie pre Docker [13]. Akákoľvek komunikácia, ako napríklad vytvorenie kontajneru alebo stiahnutie obrazu, je v skutočnosti HTTP požiadavok na konkrétny koncový bod (napríklad `POST /containers/create`). Priamy prístup cez REST API by si vyžadoval manuálnu tvorbu HTTP požiadavkov, serializáciu dát do formátu *JavaScript Object Notation* (ďalej len JSON), spracovanie HTTP odpovedí a komplexné mapovanie chybových stavov. Tento prístup je síce univerzálny, ale pre potreby tejto bakalárskej práce je nadmerne zložitý a náchylný na chyby, pričom jeho použitie neprináša nijaké značné výhody.
- **SDK:** Je vysokoúrovňová a jazykovo špecifická knižnica, ktorá obaľuje REST API do natívnych funkcií a dátových štruktúr daného programovacieho jazyka, v tomto prípade Go [14]. Pre vývojára to znamená, že namiesto manuálneho skladania HTTP požiadaviek volá bežné funkcie, pracuje s typovo bezpečnými objektmi a získava štandardné chybové typy. SDK sa stará o všetku nízkoúrovňovú komunikáciu, autentifikáciu a spracovanie dátových prúdov, čo značne zjednodušuje vývoj a zvyšuje robustnosť výslednej aplikácie.

Pre implementáciu knižnice v tejto práci bol zvolený prístup s využitím oficiálneho Go SDK pre Docker, keďže poskytuje najvyššiu mieru stability a abstrakcie.

3.2.2 Použité knižnice a technológie

Výber vhodných technológií je ovplyvnený snahou o maximálnu kompatibilitu s priemyselnými štandardmi a stabilitu výsledného rozhrania. Nasledujúce nástroje predstavujú pevný základ pre efektívnu a modernú implementáciu zadania práce.

3.2.2.1 Moby Project a oficiálne Go SDK Moby je otvorený softvérový projekt a sada komponentov, z ktorých je zložený samotný Docker. Súčasťou tohto projektu je aj oficiálna klientska knižnica pre jazyk Go. Táto knižnica je štandardom pre akúkoľvek Go aplikáciu, ktorá potrebuje komunikovať s Docker prostredím. Knižnica práve toto SDK využíva ako základ pre všetky operácie spojené so správou kontajnerov, obrazov a ďalších zdrojov. Tým je zabezpečené, že komunikácia s Docker API je spoľahlivá a konzistentná s oficiálnymi nástrojmi.

3.2.2.2 Nástroje pre budúcu rozšíriteľnosť Architektúra knižnice implementovanej v tejto práci, založená na striktnom oddelení rozhraní od implementácie, bola od začiatku navrhnutá s ohľadom na budúcu rozšíriteľnosť o podporu alternatívnych behových prostredí. Pre takéto rozšírenie by bolo možné využiť nasledujúce nástroje:

- **Podman:** Je populárne behové prostredie, ktoré poskytuje REST API kompatibilné s Docker API. Pre základnú podporu by teda bolo možné prispôbiť existujúcu implementáciu pre Docker. Pre plnohodnotnú integráciu existuje špecializovaná Go knižnica, ktorá umožňuje priamu komunikáciu s Podmanom.
- **containerd:** Je nízkoúrovňové behové prostredie, ktoré tvorí jadro samotného Dockeru, ale môže byť použité aj samostatne. Projekt containerd poskytuje vlastnú klientsku Go knižnicu, pomocou ktorej by bolo možné vytvoriť novú implementáciu rozhraní definovaných v balíčku types.

3.2.2.3 Pomocné knižnice ekosystému Go Okrem hlavného Moby SDK boli pri vývoji využité aj ďalšie špecializované knižnice, ktoré riešia špecifické implementačné detaily a prispievajú k čistote a robustnosti kódu:

- **github.com/distribution/reference:** Táto knižnica sa špecializuje na čítanie a normalizáciu názvov a referencií obrazov [15]. Zabezpečuje, že názov obrazu je správne rozložený na jeho komponenty (názov, tag, adresa registra), čo je kľúčové pre správne sťahovanie obrazov a autentifikáciu.
- **github.com/containerd/errdefs:** Knižnica poskytujúca štandardizovanú sadu typov chýb pre kontajnerové operácie (napr. NotFound, Conflict), ako aj funkcie na ich identifikáciu a spracovanie [16]. Jej použitím sa zabezpečuje konzistentné a sémanticky správne mapovanie nízkoúrovňových chýb z Docker API na chyby definované v balíčku types.

4 Návrh a implementácia riešenia

Prechod od teoretických poznatkov k praktickej realizácii vyžaduje jasnú definíciu štruktúry, ktorá zjednotí ovládanie rôznych behových prostredí pod jedno

intuitívne rozhranie. Táto časť detailne rozoberá proces transformácie stanovných požiadaviek do konkrétnych softvérových modelov, pričom sa zameriava na dosiahnutie vysokej miery oddelenia komponentov a čitateľnosti výsledného kódu.

4.1 Návrh architektúry knižnice

Architektúra knižnice bola navrhnutá s dôrazom na splnenie kľúčových nefunkčných požiadaviek definovaných v kapitole 3.1.2: jednoduchosť, rozšíriteľnosť a testovateľnosť. Tieto ciele sú dosiahnuté aplikáciou osvedčených architektonických princípov, predovšetkým princípu inverzie závislostí, ktorý bol teoreticky popísaný v kapitole 2.4. Základným kameňom celého návrhu je preto striktné oddelenie abstraktnej vrstvy (verejného API) od jej konkrétnej implementácie.

4.1.1 Štruktúra balíčkov a repozitára

Na dosiahnutie požadovanej flexibility je knižnica rozdelená do niekoľkých samostatných Go balíčkov, z ktorých každý má jasne definovanú zodpovednosť:

- **Balíček `types`:** Tento balíček predstavuje verejnú časť a centrálny kontrakt celej knižnice. Definuje všetky verejné rozhrania, ktoré knižnica obsahuje (napr. `types.Runtime`, `types.ContainerOps`), ako aj všetky dátové typy, ktoré slúžia na konfiguráciu operácií či prenos výsledkov (napríklad `types.ContainerCreateConfig`). Dôležitým architektonickým rozhodnutím je, že tento balíček neobsahuje žiadnu konkrétnu logiku a nemá žiadne externé závislosti mimo štandardnej knižnice jazyka Go. Tým je zaručené, že kód, ktorý používa túto knižnicu, závisí iba od stabilných a čistých abstrakcií.
- **Balíček `docker`:** Tento balíček poskytuje prvú a zatiaľ jedinú implementáciu rozhraní definovaných v balíčku `types`. Obsahuje internú štruktúru `runtime`, ktorá obaľuje klienta z oficiálneho Moby SDK a implementuje všetku logiku potrebnú na komunikáciu s Docker API. Používateľ knižnice s týmto balíčkom interaguje iba pri inicializácii prostredníctvom konštruktora `docker.New()`, ktorý vracia abstraktné rozhranie `types.Runtime`, čím sú všetky implementačné detaily pred používateľom skryté.
- **Použitie submodule:** Knižnica zámerne nevyužíva takzvanú techniku *reexportu*, kde by hlavný balíček exportoval typy z `types` a `docker`. Namísto toho musí používateľ explicitne importovať oba balíčky:

```
1 import (  
2     "github.com/samtaborsky/containerlib/docker"  
3     "github.com/samtaborsky/containerlib/types"  
4 )
```

Tento prístup bol zvolený z dôvodu explicitnosti a transparentnosti. Používateľ jasne vidí, že pracuje s abstraktnými typmi a žiada si konkrétnu implementáciu. Zabráňuje to skrytým závislostiam a robí architektúru čitateľnejšou.

4.1.2 Návrh verejného rozhrania

Verejné rozhranie knižnice je kompletne definované v balíčku `types` prostredníctvom Go rozhraní. Tento prístup umožňuje aplikáciám programovať voči stabilnému kontraktu a zároveň uľahčuje testovanie pomocou falošných implementácií.

Hlavným vstupným bodom je zložené rozhranie `types.Runtime`, ktoré v sebe spája niekoľko špecializovaných rozhraní rozdelených podľa princípu oddelovania rozhraní (*Interface Segregation Principle*): `SystemOps`, `ContainerOps`, `ImageOps`, `EventOps` a `TelemetryOps`.

Typický príklad použitia knižnice na vytvorenie a spustenie kontajneru je demonštrovaný v Zdrojovom kóde 1.

```
1 package main
2
3 import (
4     "context"
5     "log"
6
7     "github.com/samtaborsky/containerlib/docker"
8     "github.com/samtaborsky/containerlib/types"
9 )
10
11 func main() {
12     ctx := context.Background()
13
14     rt, err := docker.New("unix:///var/run/docker.sock")
15     if err != nil {
16         log.Fatal(err)
17     }
18     defer rt.Close()
19
20     cfg := types.ContainerCreateConfig{
21         Image: "alpine:latest",
22     }
23
24     resp, err := rt.ContainerCreate(ctx, &cfg)
25     if err != nil {
26         log.Fatal(err)
27     }
28     log.Printf("Container created with ID: %s\n", resp.ID)
29
30     if err := rt.ContainerStart(ctx, resp.ID, nil); err != nil {
31         log.Fatal(err)
32     }
33 }
```

Zdrojový kód 1: Príklad použitia knižnice na vytvorenie a spustenie kontajneru.

4.1.3 Implementačná vrstva a vzor Adaptér

Zatiaľ čo balíček `types` definuje abstraktné rozhranie knižnice, balíček `docker` poskytuje jeho konkrétnu implementačnú vrstvu pre behové prostredie Docker. V rámci architektúry plní tento balíček rolu návrhových vzorov Adaptér a Fásáda. Jeho hlavnou úlohou je úplne skryť zložitosť oficiálneho Moby SDK, ktoré priamo komunikuje s Docker API. Kým v Moby SDK musí vývojár pri operáciách zložito prepájať niekoľko rozsiahlych konfiguračných štruktúr, implementačná vrstva knižnice umožňuje použiť jedinú, zjednodušenú štruktúru (napríklad `types.ContainerCreateConfig`), z ktorej si potrebné nízkoúrovňové objekty vyskladá automaticky na pozadí.

Kľúčovým mechanizmom tejto abstrakcie sú takzvané mapovacie funkcie, ktorých úlohou je obojsmerná transformácia dátových štruktúr. Pred odoslaním požiadavky do Docker tieto funkcie konvertujú abstraktné typy z balíčka `types` na špecifické typy vyžadované Moby SDK. Pri prijatí odpovede potom dekódujú často zložité dátové štruktúry a transformujú ich do jednoduchších, presne zameraných dátových typov učených používateľovi. Vďaka tejto prísnej izolácii nie je používateľská aplikácia vôbec viazaná na vnútorné modely Dockeru, čo zabezpečuje stabilitu verejného API aj v prípade zmien v samotnom SDK.

Neoddeliteľnou súčasťou implementačnej vrstvy je aj štandardizácia spracovania chýb. Keďže Docker API vracia širokú škálu často nekonzistentných chybových stavov, implementačná vrstva ich zachytáva a prekladá na vlastnú sadu sémantických chýb definovanú v balíčku `types` (napríklad `types.ErrNotFound` alebo `types.ErrConflict`). Tento prístup zbavuje vývojára nutnosti analyzovať chybové textové reťazce a umožňuje mu priamočiaro využívať štandardnú funkciu `errors.Is()` na spoľahlivú detekciu a programatické riešenie chybových stavov.

4.2 Implementácia Docker klienta

Táto podkapitola sa venuje detailnej implementácii funkcií Docker klienta v balíčku `docker`. Kým predchádzajúce časti práce sa sústredili na abstraktný návrh, táto sekcia analyzuje, ako sú rozhrania z balíčka `types` implementované pre prostredie Docker, aké interné mechanizmy knižnica využíva a ako efektívne pracuje so súbežnosťou a transformáciou dát. Celá implementácia je zabalená v internej štruktúre `runtime`, ktorá priamo komunikuje s oficiálnym Moby SDK.

4.2.1 Inicializácia klienta a bezpečnosť vlákien

Vstupný bod balíčku `docker` je konštruktor `docker.New(host string)`, ktorý inicializuje novú instanciu klienta a používateľovi vracia objekt spĺňajúci abstraktné rozhranie `types.Runtime`. Implementácia tohto konštruktoru je navrhnutá s ohľadom na maximálnu flexibilitu pre používateľa. Ak vývojár poskytne adresu Docker API (napríklad `unix:///var/run/docker.sock`), klient sa pripojí priamo na ňu. Ak je však parameter `host` prázdny, implementá-

cia využije funkciu `client.FromEnv` z Moby SDK, ktorá automaticky načíta konfiguráciu zo štandardných premenných prostredia (ako je `DOCKER_HOST`).

Dôležitým implementačným detailom tejto štruktúry je vnútorná správa autentifikácie pre registre obrazov. Štruktúra `runtime` obsahuje internú mapu `authStore`, ktorá slúži ako pamäť pre prihlasovacie údaje a prístupové tokeny. Keď používateľ zavolá metódu `Login()`, implementácia prepošle prihlasovacie údaje Docker API a bezpečne si uloží vrátený `IdentityToken` pod doménou daného registra.

Keďže jazyk Go prirodzene podporuje súbežné vykonávanie kódu, klientska inštancia môže byť súčasne využívaná viacerými gorutinami (napríklad jedna gorutina sťahuje obraz, kým druhá sa prihlasuje do iného registru). Aby sa predišlo chybám súbežného prístupu k pamäti (*race conditions*) pri zápise a čítaní z mapy `authStore`, implementácia dôsledne chráni túto mapu pomocou synchronizačného primitíva `sync.RWMutex` (*Read-Write Mutex*) [17]. Tento prístup zaručuje, že klient je plne vláknovo bezpečný (*thread-safe*).

4.2.2 Transformácia objektov a mapovacie funkcie

Správa životného cyklu kontajnerov (vytváranie, spúšťanie, zastavovanie) priamo závisí od internej vrstvy mapovacích funkcií. Ako bolo spomenuté v kapitole 2.4, Moby SDK vyžaduje pri vytváraní kontajneru v niektorých prípadoch mimoriadne komplexnú konfiguráciu. Implementácia v balíčku `docker` túto zložitosť nielen skrýva, ale aj aktívne prepočítava a transformuje dáta do formátu, ktorému Docker rozumie.

Výborným príkladom tejto implementácie je správa a obmedzovanie hardvérových zdrojov. Docker API na najnižšej úrovni nepracuje s bežnými jednotkami (napríklad zlomky procesorových jadier alebo megabajty), ale využíva takzvané *NanoCPUs* a bajty. Vďaka interne definovaným mapovacím funkciám `toNanoCPUs` a `megabytesToBytes` môže používateľ knižnice zadať požiadavku vo formáte, ktorý je pre človeka prirodzenejší. Implementácia tieto hodnoty automaticky prepočíta (v prípade procesoru vynásobením konštantou 10^9) a správne informáciu začlení do štruktúry `container.HostConfig`, ktorú následne odošle Docker API. Tento prístup dramaticky znižuje chybovosť na strane používateľskej aplikácie.

4.2.3 Asynchrónne spracovanie telemetrie a udalostí

Najväčšou pridanou hodnotou implementácie klienta je spôsob, akým spracúva asynchrónne a nepretržité toky dát, akými sú napríklad systémové udalosti a telemetrické štatistiky bežiacich kontajnerov. Na tento účel implementácia plne využíva model súbežnosti jazyka Go, konkrétne gorutiny a kanály.

Pri funkcii `ContainerStats` klient nevráti jednu statickú hodnotu alebo dátovú štruktúru, ale vráti vlastnú dátovú štruktúru obsahujúcu dva kanály: jeden pre prichádzajúce štatistiky (`<-chan types.ContainerStats`) a druhý

pre prípadné chyby (`<-chan error`). Po zavolaní tejto metódy klient na pozadí otvorí nepretržité HTTP spojenie s Docker API a spustí novú gorutinu. Táto gorutina v nekonečnej slučke číta prichádzajúce dáta vo formáte JSON, deserializuje ich a posiela do výstupného kanála, pričom hlavné vlákno aplikácie zostáva voľné. Používateľská aplikácia potom môže tento dátový prúd elegantne a efektívne spracovávať pomocou konštrukcie `select`.

Rovnaký architektonický prístup je zvolený aj pri implementácii metódy `EventsStream`, ktorá asynchrónne počúva systémové udalosti (napríklad `die`, `start` alebo `kill`) rôznych typov a od rôznych aktérov (napríklad `container` a `image`). Implementácia navyše rešpektuje životný cyklus riadený prostredníctvom parametru `context.Context`. Ak používateľská aplikácia kontext zruší, implementácia to okamžite detekuje, bezpečne uzavrie otvorené spojenia s Docker API, ukončí gorutiny bežiace na pozadí a uzavrie všetky komunikačné kanály, čím sa predchádza únikom pamäte a gorutín.

4.3 Testovanie a validácia

Softvérová knižnica, ktorá slúži ako abstraktná vrstva nad komplexným externým systémom akým je Docker, vyžaduje dôsledné testovanie. Zlyhanie na úrovni knižnice by mohlo mať negatívny kaskádový efekt na všetky klientske aplikácie. Pre zabezpečenie spoľahlivosti a stability knižnice je testovanie rozdelené na dve hlavné úrovne: štandardné jednotkové testy vnútornej logiky a testovanie verejného rozhrania pomocou falošných (*mock*) objektov API.

4.3.1 Jednotkové testy

Jednotkové (*unit*) testy slúžia na overenie vnútornej logiky knižnice v úplnej izolácii od sieťovej komunikácie alebo externých závislostí. Ich primárnym cieľom je validovať správnosť transformácie dátových štruktúr a mapovacích funkcií, ktoré tvoria jadro implementačnej vrstvy. Pri knižnici tohto typu je veľmi dôležité pokryť kvalitnými jednotkovými testami všetky mapovacie funkcie, pretože je na nich postavená celá komunikácia s Docker API. V prípade chyby v mapovacej funkcii je dôsledkom nekonzistentný stav konfigurácie v klientskej aplikácii.

Tieto testy pokrývajú všetky mapovacie funkcie, od funkcii na konverziu hardvérových limitov (ako sú `toNanoCPUs` alebo `megabytesToBytes`), konverziu časových formátov, až po kritickú logiku mapovania chýb. Preklad chýb z nízkoúrovňového Moby SDK do sémantických chýb knižnice je pre správne fungovanie aplikácií kľúčový. Jednotkové testy v tomto prípade simulujú rôzne vstupné chybové stavy a overujú, či funkcia `mapFromMobyError` vracia očakávané typy, ako napríklad `types.ErrConflict` alebo `types.ErrNotFound`.

4.3.2 Testovanie pomocou falošných objektov

Keďže balíček `docker` priamo závisí od komunikačného klienta poskytovaného Moby SDK, na dosiahnutie izolácie od Docker API bolo nevyhnutné nahradiť

reálne sieťové pripojenie takzvaným falošným objektom (*mock object*). Knižnica využíva fakt, že oficiálne SDK definuje pre svojho klienta rozsiahle rozhranie `client.APIClient`. V testovacom prostredí bola preto vytvorená vlastná štruktúra `mockDockerAPI`, ktorá toto rozhranie implementuje, čím spĺňa kontrakt očakávaný implementačnou vrstvou.

Tento falošný objekt využíva flexibilný návrhový vzor založený na ukazovateľoch na funkcie (*function pointers*). Namiesto statického a pevne daného naprogramovania falošných odpovedí obsahuje štruktúra polia funkcií pre každú podporovanú operáciu, ako ukazuje Zdrojový kód 2.

```
1 type mockDockerAPI struct {
2     client.APIClient
3
4     containerCreateFunc func(ctx context.Context, options client.
      ContainerCreateOptions) (client.ContainerCreateResult, error)
5     imagePullFunc       func(ctx context.Context, ref string,
      options client.ImagePullOptions) (client.ImagePullResponse,
      error)
6
7     // ...
8 }
```

Zdrojový kód 2: Ukážka definície falošného klienta pre testovanie rozhrania.

Tento prístup umožňuje v každom teste dynamicky a lokálne definovať, akú presnú odpoveď alebo špecifickú chybovú hlášku má falošný klient vrátiť. Vďaka tomu je možné v testoch veľmi presne simulovať aj zriedkavé sieťové výnimky, pri ktorých je náročné vytvoriť podmienky na ich vyvolanie – napríklad zlyhanie pripojenia použitím pomocnej funkcie `generateRealConnectionError()` – bez nutnosti nadväzovať sieťové spojenie s reálnym prostredím Docker. Takéto testy poskytujú vysokú mieru istoty, že knižnica správne reaguje na akýkoľvek vstup a chybový stav zo strany Docker API.

4.4 Implementácia ukážkovej aplikácie

Aby bolo možné overiť a demonštrovať praktickú využiteľnosť, robustnosť a jednoduchosť použitia navrhutej knižnice, je v rámci práce implementovaná ukážková aplikácia. Vzhľadom na zameranie knižnice bol zvolený nástroj na automatizáciu úloh, ktorý pripomína fungovanie moderných CI/CD systémov, akými sú napríklad GitHub Actions alebo GitLab CI.

4.4.1 Návrh a princíp fungovania

Úlohou aplikácie je prijať sekvenciu po sebe idúcich krokov (*pipeline*, pričom každý krok definuje konkrétny Docker obraz a príkaz, ktorý sa má vykonať. Aplikácia tieto úlohy sekvenčne spracúva. Pre každú z nich stiahne potrebný obraz, vytvorí izolovaný dočasný kontajner, prepojí jeho štandardný a chybový výstup priamo do terminálu používateľa a následne spustí blokujúce čakanie na jeho dokončenie.

Ak proces v kontajneri skončí s nenulovou návratovou hodnotou (napríklad v prípade chyby), reťazec úloh sa okamžite a bezpečne preruší. Tento prístup je úplným základom moderného softvérového inžinierstva a automatizácie, kde sa každý test alebo zostavenie aplikácie vykonáva v čistom, izolovanom prostredí.

4.4.2 Využitie vlastností knižnice v praxi

Implementácia tejto ukážkovej aplikácie naplno demonštruje kľúčové výhody navrhutej knižnice:

- **Vysoká úroveň abstrakcie:** Kód aplikácie je lineárny a ľahko čitateľný. Vývojár nemusí ručne konfigurovať zložité štruktúry pre Docker API; rozhranie knižnice poskytuje metódy ako `ImagePull`, `ContainerCreate` či `ContainerStart` s intuitívnymi parametrami.
- **Asynchrónne spracovanie postupu operácie:** Pri sťahovaní obrazov (`ImagePull`) aplikácia demonštruje silu jazyka Go. Hlavné vlákno spustí sťahovanie, zatiaľ čo dedikovaná gorutina na pozadí asynchrónne číta udalosti z poskytnutého kanála a vizualizuje ich používateľovi.
- **Bezpečné upratovanie zdrojov (*Garbage Collection*):** Využitie štandardne používaného kľúčového slova `defer` jazyka Go v kombinácii s metódou `ContainerRemove` garantuje, že po každej úlohe sa dočasný kontajner okamžite zmaže. To platí aj v prípade nečakaného zlyhania, čím sa predchádza hromadeniu nepoužívaných kontajnerov na hostiteľskom systéme.
- **Bezpečný zber telemetrie a správa životného cyklu prúdov:** Aplikácia demonštruje pokročilú prácu s nepretržitým tokom dát prostredníctvom metódy `ContainerStats`. Štatistiky o spotrebe pamäti sú spracovávané v samostatnej gorutine pomocou konštrukcie `select`. Kľúčovým prístupom v ukážke je využitie odvodeného kontextu (`context.WithCancel`). Akonáhle hlavná úloha skončí, tento kontext je explicitne zrušený, čo dáva Docker API inštrukciu, aby okamžite prerušilo odosielanie dát. Tento detail zaručuje bezpečné ukončenie gorutiny na pozadí a efektívne zabráňuje únikom pamäte (*memory leaks*) a zablokovaní vlákien (*goroutine leaks*).
- **Spracovanie návratových hodnôt:** Aplikácia správne využíva metódu `ContainerWait` na získanie návratovej hodnoty. Keďže Docker asynchrónne zachytí koniec procesu, aplikácia naň prostredníctvom knižnice počká, získa jeho návratovú hodnotu *exit code* a na základe nej vyhodnotí úspešnosť úlohy.

Nasledujúca zjednodušená ukážka Zdrojového kódu 3 zobrazuje jadro tejto aplikácie, konkrétne funkciu `runTask`, ktorá má na starosti kompletnú orchestráciu jedného izolovaného kroku.

```

1 func runTask(ctx context.Context, rt types.Runtime, task Task) error
2     {
3         startTime := time.Now()
4
5         _ = rt.ImagePull(ctx, task.Image, nil)
6
7         createRes, _ := rt.ContainerCreate(ctx, &types.
8 ContainerCreateConfig{
9             Image: task.Image,
10            Cmd:    task.Cmd,
11        })
12
13        defer rt.ContainerRemove(context.Background(), createRes.ID, &
14 types.ContainerRemoveOptions{Force: true})
15        _ = rt.ContainerStart(ctx, createRes.ID, nil)
16
17        statsCtx, cancelStats := context.WithCancel(ctx)
18        defer cancelStats()
19        statsRes, _ := rt.ContainerStats(statsCtx, createRes.ID, &types.
20 ContainerStatsOptions{Stream: true})
21
22        avgRamChan := make(chan uint64)
23        go func() {
24            var totalRAM, count uint64
25            for {
26                select {
27                    case stats, ok := <-statsRes.Stats:
28                        if !ok {
29                            avgRamChan <- calculateAvg(totalRAM, count)
30                            return
31                        }
32                        totalRAM += stats.Memory.UsedMb
33                        count++
34                    case <-statsCtx.Done():
35                        avgRamChan <- calculateAvg(totalRAM, count)
36                        return
37                }
38            }
39        }()
40
41        _, _ = rt.ContainerLogs(ctx, createRes.ID, &types.
42 ContainerLogsOptions{
43             Stdout: os.Stdout, Stderr: os.Stderr, Follow: true,
44         })
45
46        statusRes, _ := rt.ContainerWait(ctx, createRes.ID, &types.
47 ContainerWaitOptions{})
48        if statusRes.ExitCode != 0 {
49            return fmt.Errorf("process exited with code: %d", statusRes.
50 ExitCode)
51        }
52
53        cancelStats()

```



```

47
48     fmt.Printf("[TELEMETRY] Task completed in %v | Avg RAM: %d MB",
49         time.Since(startTime).Round(time.Millisecond),
50         <-avgRamChan,
51     )
52
53     return nil
54 }

```

Zdrojový kód 3: Vykonanie izolovanej úlohy s využitím knižnice.

5 Používateľská dokumentácia

Táto kapitola slúži ako praktická príručka pre vývojárov, ktorí chcú túto knižnicu (`containerlib`) integrovať do svojich vlastných softvérových projektov. Knižnica je navrhnutá s dôrazom na jednoduchosť použitia a typovú bezpečnosť, čím výrazne zjednodušuje prácu s kontajnerovými technológiami v jazyku Go.

5.1 Predpoklady a inštalácia

Pre úspešné použitie knižnice je potrebné splniť nasledujúce systémové predpoklady:

- Nainštalované prostredie jazyka Go vo verzii 1.26 alebo novšej.
- Bežiaci Docker Engine na hostiteľskom systéme (lokálne alebo vzdialene).

Knižnicu je možné pridať do akéhokoľvek Go projektu pomocou štandardného nástroja pre správu závislostí, demonštrované v Zdrojovom kóde 4:

```
1 go get github.com/samtaborsky/containerlib
```

Zdrojový kód 4: Inštalácia knižnice.

5.2 Inicializácia a základné nastavenie

Pre prácu s knižnicou je potrebné importovať dva hlavné balíčky: `docker` pre inicializáciu konkrétneho klienta a `types` pre prácu s dátovými štruktúrami a rozhraniami.

Vstupným bodom je konštruktor `docker.New()`. Tento konštruktor prijíma ako parameter adresu hostiteľa. Ak je parameter prázdny, klient sa automaticky nakonfiguruje podľa štandardných premenných prostredia. Po ukončení práce je nevyhnutné klienta bezpečne uzavrieť pomocou metódy `Close()`. Inicializáciu ukazuje Zdrojový kód 5.

```

1 import (
2     "log"
3
4     "github.com/samtaborsky/containerlib/docker"
5     "github.com/samtaborsky/containerlib/types"
6 )
7
8 func main() {
9     // Initialization using environment variables
10    rt, err := docker.New("")
11    // Alternatively: docker.New("unix:///var/run/docker.sock")
12
13    if err != nil {
14        log.Fatalf("Could not connect: %v", err)
15    }
16    defer rt.Close()
17 }

```

Zdrojový kód 5: Inicializácia klienta.

5.3 Práca s Docker obrazmi

Knižnica poskytuje metódy na sťahovanie, inšpekciu, výpis a odstránenie obrazov. Najčastejšie používanou operáciou je sťahovanie obrazu (`ImagePull`). Voliteľný parameter `types.ImagePullOptions` umožňuje nastaviť dodatočné parametre, ako napríklad odber asynchrónneho prúdu udalostí o postupe sťahovania prostredníctvom Go kanálu. Použitie tejto funkcionality je demonštrované v Zdrojovom kóde 6

```

1 err := rt.ImagePull(ctx, "alpine:latest", nil)
2 if err != nil {
3     log.Fatalf("Error while pulling image: %v", err)
4 }

```

Zdrojový kód 6: Ukážka stiahnutia obrazu.

5.4 Správa životného cyklu kontajnerov

Vytvorenie a spustenie kontajneru je vďaka knižnici veľmi jednoduché. Vývojár definuje požiadavky na vytvorenie kontajneru prostredníctvom konfiguračnej štruktúry `types.ContainerCreateConfig`. Nasledujúci Zdrojový kód 7 demonštruje správu kompletného životného cyklu: vytvorenie, spustenie a blokujúce čakanie na dokončenie procesu.

```

1 // 1. Create
2 cfg := types.ContainerCreateConfig{
3     Image: "alpine:latest",
4     Cmd:   []string{"echo", "Hello world!"},
5 }
6
7 res, err := rt.ContainerCreate(ctx, &cfg)
8 if err != nil {
9     log.Fatal(err)
10 }
11
12 // Make sure to delete the container at the end
13 defer rt.ContainerRemove(context.Background(), res.ID, nil)
14
15 // 2. Start
16 if err := rt.ContainerStart(ctx, res.ID, nil); err != nil {
17     log.Fatal(err)
18 }
19
20 // 3. Wait for exit and return exit code
21 res, err = rt.ContainerWait(ctx, res.ID, nil)
22 if err != nil {
23     log.Fatal(err)
24 }
25 log.Printf("Container exited with code: %d.", res.ExitCode)

```

Zdrojový kód 7: Ukážka správy životného cyklu kontajneru.

5.5 Odber logov a telemetrie

Knižnica využíva na streamovanie dát kanály a štandardné rozhrania jazyka Go. Pre získanie logov z kontajneru stačí v možnostiach definovať instance typu `io.Writer` (napríklad priamo štandardný výstup `os.Stdout`).

Pre odber telemetrických dát (štatistik využitia procesoru a operačnej pamäti) knižnica vracia objekt obsahujúci dátový a chybový kanál, ktoré je možné jednoducho spracovávať pomocou konštrukcie `select`, ako v Zdrojovom kóde 8.

```

1 opts := types.ContainerStatsOptions{Stream: true}
2
3 res, err := rt.ContainerStats(ctx, containerID, &opts)
4 if err != nil {
5     log.Fatal(err)
6 }
7
8 for {
9     select {
10        case stats, ok := <-res.Stats:
11            if !ok {
12                log.Println("Channel was closed.")
13                return
14            }

```

```

15         log.Printf("Memory used: %d MB.", stats.Memory.UsedMb)
16
17     case err, ok := <-res.Errors:
18         if !ok {
19             return
20         }
21         log.Fatalf("Error while streaming: %v", err)
22     }
23 }

```

Zdrojový kód 8: Ukážka čítania telemetrie v reálnom čase.

5.6 Spracovanie chýb

Implementačná vrstva knižnice prekladá nízkoúrovňové chyby z Docker API na vlastnú sadu štandardizovaných sémantických chýb definovaných v balíčku `types`. To umožňuje vývojárom robustne a presne reagovať na špecifické chybové stavy pomocou štandardnej funkcie `errors.Is()`, napríklad ako v Zdrojovom kóde 9.

```

1 err := rt.ContainerStart(ctx, "non-existent-id", nil)
2
3 if err != nil {
4     if errors.Is(err, types.ErrNotFound) {
5         log.Println("Provided container was not found.")
6     } else if errors.Is(err, types.ErrConflict) {
7         log.Println("Provided container has a state conflict.")
8     } else {
9         log.Printf("Unknown error: %v", err)
10    }
11 }

```

Zdrojový kód 9: Ukážka spracovania chýb.

6 Možné budúce rozšírenia

Hoci aktuálna implementácia knižnice úspešne spĺňa stanovené ciele a požiadavky tejto bakalárskej práce, jej architektúra bola od samého začiatku navrhovaná s ohľadom na dlhodobú udržiateľnosť a možnosť postupného rozširovania. Striktné oddelenie abstraktných rozhraní od konkrétnej implementácie poskytuje ideálny základ pre pridávanie novej funkcionality bez narušenia existujúceho klientskeho kódu. V nasledujúcich podkapitolách sú zhrnuté hlavné smery, ktorými by som chcel vývoj projektu smerovať v budúcnosti.

6.1 Podpora ďalších behových prostredí

Najprirodzenejším a najdôležitejším rozšírením knižnice bude implementácia podpory pre ďalšie kontajnerové behové prostredia. V súčasnosti knižnica obsahuje

iba balíček `docker`. Vďaka rozhraniu `types.Runtime` je však systém pripravený na integráciu nasledujúcich alternatív:

- **Podman:** Keďže Podman poskytuje REST API, ktoré je z veľkej časti kompatibilné s Docker API, pridanie balíčka `podman` by si vyžadovalo implementáciu špecifického klienta, ktorý by však mohol čiastočne zdieľať logiku mapovania s existujúcim riešením. Tým by knižnica získala podporu pre čoraz populárnejší prístup ku kontajnerom (*daemonless* behové prostredia).
- **containerd:** Ako nízkoúrovňové jadro, na ktorom je postavený aj samotný Docker, poskytuje containerd vlastné gRPC API a Go klientsku knižnicu. Vytvorenie implementačnej vrstvy a balíčka `containerd` by umožnilo vývojárom spravovať kontajnery s absolútne minimálnou systémovou réžiou, čo je ideálne pre okrajové zariadenia (*edge computing*) a vysoko optimalizované systémy.

6.2 Správa zväzkov a sieťovanie

Aktuálna verzia verejného API v balíčku `types` sa zameriava primárne na životný cyklus kontajnerov, obrazov, telemetriu a udalosti. Pre plnohodnotné nasadenie v komplexných distribuovaných systémoch je však nevyhnutné abstrahovať aj ďalšie zdroje:

- **Správa zväzkov (Volumes):** Vytvorenie rozhrania `VolumeOps`, ktoré by zastrešovalo správu perzistentného úložiska. To zahŕňa operácie ako vytvorenie zväzku, jeho odstránenie, výpis a inšpekciu, čo by umožnilo aplikáciám bezpečne spravovať stavové dáta nezávisle od životného cyklu samotného kontajneru.
- **Správa sietí (Networking):** Vytvorenie rozhrania `NetworkOps` na programové vytváranie virtuálnych sietí (napríklad typu *bridge* alebo *overlay*), pripájanie kontajnerov do týchto sietí a správu izolácie komunikácie medzi viacerými bežiacimi službami.

6.3 Open-source pokračovanie

Odovzdaním tejto bakalárskej práce sa vývoj knižnice nekončí. Mojm osobným plánom a ambíciou je sa tomuto projektu aktívne venovať aj po ukončení štúdia. Knižnica bude naďalej verejne dostupná ako open-source projekt pod *MIT* licenciou [18].

Cieľom tohto dlhodobého pokračovania je priviesť knižnicu do stavu blízkeho plnej produkčnej použiteľnosti a vysokej bezpečnosti. To zahŕňa:

- Stabilizáciu verejného API a prípravu na vydanie verzie `1.0.0` s garanciou spätnej kompatibility.

- Vykonanie hĺbkových bezpečnostných auditov kódu a optimalizáciu správy pamäte pri práci s veľkými dátovými prúdmi (telemetria, logy).
- Zapojenie širšej vývojárskej komunity prostredníctvom platformy GitHub, prijímanie návrhov na vylepšenia a riešenie nahlásených chýb.

Týmto pokračovaním má projekt ambíciu stať sa spoľahlivým, moderným a pohodlne použiteľným štandardom pre vývojárov v jazyku Go, ktorí potrebujú vo svojom kóde manipulovať s kontajnerovými technológiami nezávisle od behového prostredia, ktoré sa nachádza na systéme.

Záver

Cieľom tejto práce bolo navrhnuť a implementovať rozšíriteľnú softvérovú knižnicu v jazyku Go pre správu Docker kontajnerov. Funkcionalita mala obsiahnuť kompletný životný cyklus kontajnerov, prácu s obrazmi, ako aj sledovanie udalostí a telemetrie. Návrh API mal byť predovšetkým intuitívny, typovo bezpečný a mal skryť nízkoúrovňovú komplexnosť oficiálneho SDK. Tvorba knižnice určenej pre iných vývojárov kládla dôraz na čistý návrh rozhraní a striktné oddelenie abstrakcie od samotnej implementácie.

Výsledná knižnica spĺňa definované požiadavky. Predstavuje nielen plnohodnotný a otestovaný softvérový komponent, ale aj stabilný základ pre ďalší open-source vývoj a rozšírenie o iné behové prostredia, ako napríklad Podman. Riešenie pokrýva všetky kľúčové aspekty modernej Go knižnice – od čistého návrhu dátových štruktúr, cez bezpečné asynchrónne spracovanie dátových prúdov pomocou gorutín a kanálov, až po spoľahlivé spracovanie chýb.

Vývoj tohto projektu bol pre mňa nepochybne prínosom a výbornou prípravou pre prax v softvérovom inžinierstve. Implementácia takto komplexného nástroja mi jasne ukázala, aké dôležité je mať vopred premyslenú architektúru a vyriešené otázky návrhových vzorov pred samotným začiatkom vývoja. Zároveň mi to pomohlo lepšie odhadnúť náročnosť tvorby stabilného kódu, ktorý je nielen funkčný, ale aj dlhodobo udržiateľný.

Conclusions

The aim of this thesis was to design and implement an extensible software library in the Go programming language for Docker container management. The functionality was to encompass the complete container lifecycle, image management, as well as event and telemetry monitoring. The API design was primarily intended to be intuitive, type-safe, and hide the low-level complexity of the official SDK. Creating a library intended for other developers placed an emphasis on clean interface design and the strict separation of abstraction from the implementation itself.

The resulting library fulfills the defined requirements. It represents not only a fully-fledged and tested software component, but also a stable foundation for further open-source development and expansion to other runtimes, such as Podman. The solution covers all key aspects of a modern Go library—from the clean design of data structures to safe asynchronous processing of data streams using goroutines and channels, and reliable error handling.

Developing this project was undoubtedly of benefit to me and an excellent preparation for software engineering practice. Implementing such a complex tool clearly showed me the importance of having a well-thought-out architecture and resolved design patterns before the actual development begins. At the same time, it helped me better estimate the effort required to create a stable codebase that is not only functional but also maintainable in the long term.

A Obsah elektronických dát

Elektronické dáta tejto bakalárskej práce sú rozdelené do štruktúry adresárov a súborov takto:

README.txt

Textový súbor s informáciami o opakovateľnom spôsobe použitia ostatných dát práce, systémové predpoklady a spôsob spojazdnenia.

source.zip

ZIP súbor obsahujúci kompletný zdrojový kód knižnice a ukázkovej aplikácie.

text/

Adresár s textom práce vo formáte PDF a ZIP archívom, ktorý obsahuje všetky súbory potrebné pre bezproblémové vytvorenie PDF dokumentu textu, tj. zdrojový text textu a príloh, vložené obrázky, apod.

Pri všetkých cudzích obsiahnutých materiáloch ich zahrnutie dovoľujú podmienky na ich verejné šírenie alebo priložený súhlas držiteľa práv na použitie. Pre všetky použité (a citované) materiály, pri ktorých toto nie je splnené a nie sú tak obsiahnuté, je uvedený ich zdroj, napr. webová adresa, v bibliografii alebo texte práce alebo v súbore README.txt.

Literatúra

- [1] MOBY PROJECT. Moby Project [online]. [cit. 2026-04-29]. Dostupné z: <https://mobyproject.org/>
- [2] PODMAN PROJECT. Podman Go Bindings [online]. [cit. 2026-04-29]. Dostupné z: <https://github.com/containers/podman/tree/main/pkg/bindings>
- [3] CONTAINERD PROJECT. containerd [online]. [cit. 2026-04-29]. Dostupné z: <https://github.com/containerd/containerd>
- [4] KUBERNETES. Container Runtime Interface (CRI) [online]. 12-02-2026. [cit. 2026-04-29]. Dostupné z: <https://kubernetes.io/docs/concepts/containers/cri/>
- [5] TESTCONTAINERS. Testcontainers [online]. [cit. 2026-04-29]. Dostupné z: <https://testcontainers.com/>
- [6] DOCKER. Docker overview [online]. [cit. 2026-04-29]. Dostupné z: <https://docs.docker.com/get-started/docker-overview/>
- [7] DOCKER. What is a container? [online]. [cit. 2026-04-29]. Dostupné z: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [8] DOCKER. What is an image? [online]. [cit. 2026-04-29]. Dostupné z: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>
- [9] DOCKER. What is a registry? [online]. [cit. 2026-04-29]. Dostupné z: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-registry/>
- [10] HOARE, C. A. R. *Communicating Sequential Processes* [online]. 2004 [cit. 2026-05-12]. Dostupné z: <https://www.cs.ox.ac.uk/ucs/hoarebook.pdf>
- [11] COX-BUDAY, K. *Concurrency in Go: Tools and Techniques for Developers* [online]. O'Reilly Media, 2017 [cit. 2026-04-29]. ISBN 978-1491941195.
- [12] MARTIN, ROBERT. *Clean architecture: A Craftsman's Guide to Software Structure and Design*. Pearson Professional, 2018 [cit. 2026-04-29]. ISBN 978-0134494166.
- [13] DOCKER. Docker Engine API [online]. [cit. 2026-04-29]. Dostupné z: <https://docs.docker.com/reference/api/engine/>
- [14] MOBY PROJECT. Go client for the Docker Engine API [online]. [cit. 2026-04-29]. Dostupné z: <https://pkg.go.dev/github.com/moby/moby/client>

- [15] CLOUD NATIVE COMPUTING FOUNDATION. Distribution reference [online]. [cit. 2026-04-29]. Dostupné z: <https://github.com/distribution/reference>
- [16] CONTAINERD PROJECT. Common definition and library of errors used by containerd [online]. [cit. 2026-04-29]. Dostupné z: <https://github.com/containerd/errdefs>
- [17] THE GO AUTHORS. sync package - RWMutex - Go Packages [online]. [cit. 2026-05-04]. Dostupné z: <https://pkg.go.dev/sync#RWMutex>
- [18] OPEN SOURCE INITIATIVE. The MIT License [online]. [cit. 2026-05-04]. Dostupné z: <https://opensource.org/licenses/MIT>